Présentations, Sheets et Alertes en SwiftUI

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'une présentation modale ?

Une présentation modale permet d'afficher temporairement une interface au-dessus de l'écran courant, sans modifier nécessairement la navigation principale de l'application.

SwiftUI fournit notamment :

```
.sheet
.fullScreenCover
.alert
.confirmationDialog
```

Ces présentations sont pilotées par l'état : lorsque la valeur associée change, SwiftUI affiche ou ferme automatiquement la présentation.

2. Navigation ou présentation modale ?

Avec NavigationStack :

```
Liste
  ↓
Détail
  ↓
Sous-détail
```

On avance dans une hiérarchie.

Avec une présentation modale :

```
Écran courant
      ↓
    Sheet
```

La nouvelle interface apparaît temporairement par-dessus l'écran courant.

3. Quand utiliser une présentation modale ?

Exemples :

- Modifier un profil
- Ajouter un élément
- Afficher un formulaire
- Choisir une option
- Confirmer une opération
- Afficher une information importante

4. sheet

Une sheet affiche une vue de manière modale.

On contrôle généralement son affichage avec :

```
@State
```

Exemple :

```
@State private var afficherProfil = false
```

Puis :

```
.sheet(isPresented: $afficherProfil) {
    ProfilView()
}
```

sheet(isPresented:) utilise un Binding<Bool> pour déterminer si la feuille doit être présentée.

5. Premier exemple

```
struct ContentView: View {
    @State private var afficherProfil = false

    var body: some View {
        Button("Modifier le profil") {
            afficherProfil = true
        }
        .sheet(
            isPresented: $afficherProfil
        ) {
            ProfilView()
        }
    }
}
```

Lorsque afficherProfil = true, la feuille apparaît.

6. Vue présentée

```
struct ProfilView: View {
    var body: some View {
        VStack(spacing: 20) {
            Text("Modifier le profil")
                .font(.title)
            TextField(
                "Nom",
                text: .constant("")
            )
            .textFieldStyle(.roundedBorder)
        }
        .padding()
    }
}
```

7. Fermer une sheet

Une vue présentée peut accéder à :

```
@Environment(\.dismiss)
```

Puis appeler :

```
dismiss()
```

dismiss permet de fermer la présentation courante, notamment une sheet.

8. Exemple avec dismiss

```
struct ProfilView: View {
    @Environment(\.dismiss)
    private var dismiss

    var body: some View {
        VStack(spacing: 20) {
            Text("Modifier le profil")
            Button("Fermer") {
                dismiss()
            }
        }
    }
}
```

9. Exemple plus réaliste

```
struct ModifierProfilView: View {
    @Environment(\.dismiss)
    private var dismiss
    @State private var nom = ""

    var body: some View {
        NavigationStack {
            Form {
                TextField(
                    "Nom",
                    text: $nom
                )
            }
            .navigationTitle(
                "Modifier le profil"
            )
            .toolbar {
                ToolbarItem(
                    placement: .cancellationAction
                ) {
                    Button("Annuler") {
                        dismiss()
                    }
                }
                ToolbarItem(
                    placement: .confirmationAction
                ) {
                    Button("Enregistrer") {
                        dismiss()
                    }
                }
            }
        }
    }
}
```

10. onDismiss

Il est aussi possible d'exécuter du code lorsque la feuille est fermée.

```
.sheet(
    isPresented: $afficherProfil,
    onDismiss: {
        print("La feuille est fermée")
    }
) {
    ProfilView()
}
```

La variante sheet(isPresented:onDismiss:content:) permet d'associer une action à la fermeture.

11. interactiveDismissDisabled

Par défaut, l'utilisateur peut fermer une sheet en la faisant glisser vers le bas.

Lorsqu'un formulaire ne doit pas être fermé accidentellement, on peut utiliser :

```
.interactiveDismissDisabled()
```

Exemple :

```
ModifierProfilView()
    .interactiveDismissDisabled()
```

12. Présenter une donnée avec sheet(item:)

Une sheet peut aussi être directement associée à une donnée optionnelle.

Exemple :

```
struct Produit: Identifiable {
    let id = UUID()
    let nom: String
}
```

Puis :

```
@State
private var produitSelectionne: Produit?
```

13. Exemple avec sheet(item:)

```
List(produits) { produit in
    Button(produit.nom) {
        produitSelectionne = produit
    }
}
.sheet(
    item: $produitSelectionne
) { produit in
    DetailProduitView(
        produit: produit
    )
}
```

La feuille est présentée lorsqu'un élément non nil est fourni, et l'élément est transmis au contenu présenté.

14. Pourquoi utiliser sheet(item:) ?

Au lieu de gérer séparément :

```
@State private var afficherSheet = false
@State private var produitSelectionne: Produit?
```

on peut simplement utiliser :

```
@State private var produitSelectionne: Produit?
```

Cela relie directement :

```
la donnée sélectionnée
        ↓
    la présentation
```

15. Taille d'une sheet

Une feuille peut avoir différentes hauteurs grâce à :

```
.presentationDetents(...)
```

Par exemple :

```
.sheet(
    isPresented: $afficherSheet
) {
    ProfilView()
        .presentationDetents([
            .medium,
            .large
        ])
}
```

presentationDetents définit les hauteurs auxquelles la feuille peut se positionner. Avec plusieurs valeurs, l'utilisateur peut la redimensionner entre ces positions.

La feuille peut alors occuper approximativement :

Valeur	Hauteur approximative
`.medium`	Une partie de l'écran
`.large`	Une grande partie de l'écran

16. Hauteur personnalisée

On peut aussi utiliser :

```
.presentationDetents([
    .height(250),
    .large
])
```

ou une fraction :

```
.presentationDetents([
    .fraction(0.4),
    .large
])
```

17. Indicateur de glissement

Une feuille peut afficher un indicateur permettant de montrer qu'elle peut être déplacée.

```
.presentationDragIndicator(
    .visible
)
```

presentationDragIndicator contrôle la visibilité de cet indicateur sur une feuille.

18. Exemple complet de sheet

```
struct ContentView: View {
    @State
    private var afficherParametres = false

    var body: some View {
        Button("Paramètres") {
            afficherParametres = true
        }
        .sheet(
            isPresented: $afficherParametres
        ) {
            ParametresView()
                .presentationDetents([
                    .medium,
                    .large
                ])
                .presentationDragIndicator(
                    .visible
                )
        }
    }
}
```

19. fullScreenCover

Dans certains cas, on souhaite que la présentation occupe pratiquement tout l'écran.

On utilise :

```
.fullScreenCover
```

SwiftUI fournit des variantes basées sur un Bool ou sur un élément optionnel.

20. Exemple — Écran de connexion en plein écran

```
struct ContentView: View {
    @State
    private var afficherConnexion = false

    var body: some View {
        Button("Connexion") {
            afficherConnexion = true
        }
        .fullScreenCover(
            isPresented: $afficherConnexion
        ) {
            ConnexionView()
        }
    }
}
```

21. Fermer un fullScreenCover

Même principe :

```
struct ConnexionView: View {
    @Environment(\.dismiss)
    private var dismiss

    var body: some View {
        VStack {
            Text("Connexion")
            Button("Fermer") {
                dismiss()
            }
        }
    }
}
```

22. sheet ou fullScreenCover ?

sheet

Appropriée pour :

- Modifier une information
- Ajouter un élément
- Afficher des paramètres
- Présenter un formulaire relativement court

fullScreenCover

Approprié lorsque :

- L'expérience doit occuper tout l'écran
- Plusieurs actions importantes sont nécessaires
- L'utilisateur doit se concentrer sur une tâche

23. Les alertes

Une alerte permet d'attirer rapidement l'attention sur une information ou une décision importante.

En SwiftUI :

```
.alert
```

Les alertes font partie des présentations modales proposées par SwiftUI.

24. Première alerte

```
struct ContentView: View {
    @State
    private var afficherAlerte = false

    var body: some View {
        Button("Supprimer") {
            afficherAlerte = true
        }
        .alert(
            "Suppression",
            isPresented: $afficherAlerte
        ) {
            Button("OK") {
            }
        }
    }
}
```

25. Ajouter un message

```
.alert(
    "Suppression",
    isPresented: $afficherAlerte
) {
    Button("OK") {
    }
} message: {
    Text(
        "L'élément sera supprimé."
    )
}
```

26. Plusieurs boutons

```
.alert(
    "Supprimer le produit ?",
    isPresented: $afficherAlerte
) {
    Button(
        "Supprimer",
        role: .destructive
    ) {
        supprimerProduit()
    }
    Button(
        "Annuler",
        role: .cancel
    ) {
    }
} message: {
    Text(
        "Cette action est irréversible."
    )
}
```

27. Rôle des boutons

SwiftUI permet d'associer certains rôles aux boutons.

Rôle	Utilisation
`role: .destructive`	Pour une action destructive
`role: .cancel`	Pour une annulation

Exemple typique :

```
Supprimer
Annuler
```

28. Une alerte pilotée par l'état

Le principe reste le même que pour une sheet.

```
@State
private var afficherAlerte = false
```

Puis :

```
afficherAlerte = true
```

SwiftUI met à jour l'interface et affiche l'alerte.

29. Exemple — Ajouter un produit

```
struct AjouterProduitView: View {
    @State private var nom = ""
    @State
    private var afficherConfirmation = false

    var body: some View {
        VStack(spacing: 20) {
            TextField(
                "Nom du produit",
                text: $nom
            )
            .textFieldStyle(.roundedBorder)
            Button("Ajouter") {
                afficherConfirmation = true
            }
        }
        .padding()
        .alert(
            "Produit ajouté",
            isPresented:
            $afficherConfirmation
        ) {
            Button("OK") {
            }
        } message: {
            Text(
                "\(nom) a été ajouté."
            )
        }
    }
}
```

30. confirmationDialog

Lorsqu'on souhaite proposer plusieurs actions à l'utilisateur, SwiftUI fournit :

```
.confirmationDialog
```

Il s'agit également d'un type de présentation modale.

31. Exemple — Choisir une action

```
struct ContentView: View {
    @State
    private var afficherActions = false

    var body: some View {
        Button("Actions") {
            afficherActions = true
        }
        .confirmationDialog(
            "Choisir une action",
            isPresented: $afficherActions
        ) {
            Button("Modifier") {
                print("Modifier")
            }
            Button(
                "Supprimer",
                role: .destructive
            ) {
                print("Supprimer")
            }
            Button(
                "Annuler",
                role: .cancel
            ) {
            }
        }
    }
}
```

32. alert ou confirmationDialog ?

alert

À utiliser principalement pour :

- Informer
- Confirmer une action importante
- Signaler une erreur
- Demander une confirmation simple

confirmationDialog

À utiliser lorsque plusieurs actions sont possibles :

- Modifier
- Partager
- Dupliquer
- Supprimer

33. Exemple — Gestion d'un produit

```
Button("Options") {
    afficherOptions = true
}
.confirmationDialog(
    "Actions sur le produit",
    isPresented: $afficherOptions
) {
    Button("Modifier") {
        afficherModification = true
    }
    Button("Partager") {
        partager()
    }
    Button(
        "Supprimer",
        role: .destructive
    ) {
        afficherSuppression = true
    }
}
```

34. Combiner confirmationDialog et alert

Une application peut enchaîner :

```
confirmationDialog
        ↓
choisir Supprimer
        ↓
       alert
        ↓
confirmer définitivement
```

Exemple :

```
.confirmationDialog(
    "Actions",
    isPresented: $afficherOptions
) {
    Button(
        "Supprimer",
        role: .destructive
    ) {
        afficherAlerteSuppression = true
    }
}
.alert(
    "Confirmer la suppression",
    isPresented: $afficherAlerteSuppression
) {
    Button(
        "Supprimer",
        role: .destructive
    ) {
        supprimer()
    }
    Button(
        "Annuler",
        role: .cancel
    ) {
    }
}
```

35. Présentation et données

Les présentations fonctionnent très bien avec les notions déjà vues :

```
@State
@Binding
@Observable
MVVM
```

Par exemple, @State private var afficherAjout = false contrôle l'interface. Mais la logique réelle peut être placée dans un ViewModel.

36. Exemple avec MVVM

```
@Observable
class ProduitsViewModel {
    var produits: [Produit] = []

    func supprimer(
        _ produit: Produit
    ) {
        produits.removeAll {
            $0.id == produit.id
        }
    }
}
```

La vue peut gérer l'affichage de l'alerte, et le ViewModel la suppression du produit.

37. Responsabilité de la View

La View gère principalement :

- Sheet ouverte ou fermée
- Alerte affichée ou non
- Élément sélectionné
- Dialogue affiché

Exemple :

```
@State
private var afficherSheet = false
@State
private var afficherAlerte = false
```

38. Responsabilité du ViewModel

Le ViewModel gère plutôt :

- Ajouter
- Modifier
- Supprimer
- Valider
- Charger les données

Exemple :

```
viewModel.supprimer(produit)
```

39. Exemple d'organisation

```
ProduitsView
     │
     ├── sheet
     │      └── AjouterProduitView
     │
     ├── confirmationDialog
     │      ├── Modifier
     │      └── Supprimer
     │
     └── alert
            └── Confirmation suppression
```

40. Exemple complet simplifié

Cet exemple combine les trois présentations illustrées ci-dessus.

```
struct ProduitsView: View {
    @State
    private var afficherAjout = false
    @State
    private var afficherOptions = false
    @State
    private var afficherAlerte = false

    var body: some View {
        VStack(spacing: 20) {
            Button("Ajouter") {
                afficherAjout = true
            }
            Button("Options") {
                afficherOptions = true
            }
        }
        .sheet(
            isPresented: $afficherAjout
        ) {
            AjouterProduitView()
        }
        .confirmationDialog(
            "Actions sur le produit",
            isPresented: $afficherOptions
        ) {
            Button("Modifier") {
            }
            Button(
                "Supprimer",
                role: .destructive
            ) {
                afficherAlerte = true
            }
        }
        .alert(
            "Supprimer ?",
            isPresented: $afficherAlerte
        ) {
            Button(
                "Supprimer",
                role: .destructive
            ) {
            }
            Button(
                "Annuler",
                role: .cancel
            ) {
            }
        }
    }
}
```

Choisir Supprimer dans le dialogue déclenche ensuite l'alerte de confirmation.

41. Attention à dismiss

dismiss doit être récupéré dans l'environnement de la vue réellement présentée — et non dans la vue qui déclenche la présentation.

```
ProduitsView  (présente la sheet)
        │
        │  .sheet { AjouterProduitView() }
        ▼
AjouterProduitView  (vue présentée)
        │
        └── @Environment(\.dismiss) doit être lu ICI
```

L'action dismiss dépend de l'environnement où elle est lue.

42. Bonnes pratiques

Éviter de multiplier les variables comme :

```
afficherSheet1
afficherSheet2
afficherSheet3
afficherSheet4
```

lorsque la présentation dépend réellement d'une donnée. Dans ce cas, une approche comme :

```
.sheet(item:)
```

peut être plus claire.

43. Choisir la bonne présentation

Outil	Utilisation
`NavigationLink`	Aller vers un autre écran de la hiérarchie
`sheet`	Tâche temporaire ou formulaire
`fullScreenCover`	Tâche modale occupant tout l'écran
`alert`	Message ou confirmation courte
`confirmationDialog`	Choix parmi plusieurs actions

44. Résumé

sheet

```
.sheet(
    isPresented: $afficher
) {
    MaView()
}
```

Fermer

```
@Environment(\.dismiss)
private var dismiss
dismiss()
```

Plein écran

```
.fullScreenCover(
    isPresented: $afficher
) {
    MaView()
}
```

Alerte

```
.alert(
    "Titre",
    isPresented: $afficher
) {
    Button("OK") {
    }
}
```

Dialogue d'actions

```
.confirmationDialog(
    "Actions",
    isPresented: $afficher
) {
    // boutons
}
```

45. Exercice — Gestion d'un profil

Créer une vue :

```
ProfilView
```

contenant :

- Nom
- Courriel
- Bouton Modifier
- Bouton Actions

1. Sheet

Lorsque l'utilisateur appuie sur Modifier, ouvrir :

```
ModifierProfilView
```

avec :

```
.sheet
```

Le formulaire doit contenir nom et courriel, et les boutons Annuler / Enregistrer. Utiliser :

```
dismiss()
```

pour fermer la feuille.

2. Taille de la sheet

Configurer la feuille avec :

```
.presentationDetents([
    .medium,
    .large
])
```

et :

```
.presentationDragIndicator(
    .visible
)
```

3. Actions

Le bouton Actions doit afficher un :

```
confirmationDialog
```

avec :

- Partager
- Déconnexion
- Supprimer le profil

4. Alerte

Lorsque Supprimer le profil est sélectionné, afficher une :

```
alert
```

demandant :

```
Console
Voulez-vous vraiment supprimer le profil ?
```

avec deux choix : Supprimer et Annuler.

46. À retenir

Les présentations SwiftUI reposent principalement sur l'état :

```
État modifié
      ↓
SwiftUI met à jour l'interface
      ↓
Présentation affichée ou fermée
```

Les outils principaux sont :

```
.sheet
.sheet(item:)
.fullScreenCover
.alert
.confirmationDialog
dismiss()
.presentationDetents
.presentationDragIndicator
```